ADAPTQUANT: HARDWARE-ALIGNED PER-TILE MX FORMAT SELECTION FOR POST-TRAINING QUANTIZATION ON NVIDIA BLACKWELL

Anonymous authors
Paper under double-blind review

ABSTRACT

Post-Training Quantization cuts the cost of serving Large Language Models, and NVIDIA Blackwell's native support for microscaling (MX) formats pushes the speedup further. Yet uniform MXFP4 still incurs substantial quantization error, and existing fixes assign higher-precision formats to sensitive regions using calibration data: a cost repeated for every new model before it can be served at all. We propose AdaptQuant, which assigns an MX element format per 16×32 tile, the granularity at which Blackwell's block-scaled MMA reads its operands, from a single statistic computed in one pass over the tile. The rule mapping this statistic to a format is derived once, offline, on a single reference model against NVFP4 and reused unchanged on every other model. Across 13 open-weight LLMs on a single RTX 5090, AdaptQuant raises mean perplexity over BF16 by only 5.0% on WikiText-2 and 3.9% on C4 while spending fewer effective weight bits on average than the strongest mixed-precision baseline and quantizing $2.9\times$ faster, cutting stored weight memory by up to $3.08\times$ over BF16 and reaching $1.81\times$ BF16's prefill throughput.

1 INTRODUCTION

Serving large language models (LLMs) at scale is dominated by the cost of storing their weights and moving them: memory and bandwidth set the latency, the energy consumption, and the amount of hardware a deployment requires. Post-Training Quantization (PTQ) is the standard remedy, shrinking weights and, in more aggressive schemes, activations, to fewer bits without retraining (Frantar et al., 2023; Xiao et al., 2023; Dettmers et al., 2022). Turning this storage saving into an actual speedup, however, requires hardware that computes natively at low precision: on earlier GPUs, a quantized value is dequantized before each matrix multiply, so throughput tracks the higher precision regardless of the stored width. NVIDIA's Blackwell is the first architecture to close this gap, executing block-scaled matrix multiplies at native throughput (NVIDIA Corporation, 2025) – but only for values actually stored at the narrowest width, which is precisely the accuracy burden that any block-scaled format must carry.

Two block-scaled families currently have native instruction support. The Open Compute Project's Microscaling (MX) standard (Rouhani et al., 2023) fixes a 32-element block under a power-of-two (E8M0) scale; NVIDIA's NVFP4 (Alvarez et al., 2025) pairs a finer 16-element block with an E4M3 scale. This coarser block and scale come at the cost: at the same 4-bit element width, MXFP4 loses more accuracy than NVFP4. Because a quantization method built on the OCP MX standard cannot change the block size or scale format, any accuracy it recovers must come from the element format alone.

Mixed-precision methods recover the accuracy MXFP4 loses by varying the element format across a tensor, but every existing method pays a calibration cost for this flexibility. MicroMix (Liu et al., 2026), the closest prior work, assigns each activation channel a format from calibrated thresholds and a channel reordering, refitting both on every new model. This dependence is not specific to mixing formats: ARCQuant (Meng et al., 2026) pays the same price while quantizing uniformly to NVFP4. Figure 1 measures this cost on an RTX 5090, and it grows with model size: from Llama-3.1-8B to Qwen2.5-32B, MicroMix's preparation time rises by $2.9\times$, ARCQuant's by $3.1\times$, GPTQ's by $4.4\times$,

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

and AWQ's (Lin et al., 2024) by $4.1\times$. Every model deployed pays this cost before serving a single token, and none of it carries over to the next.

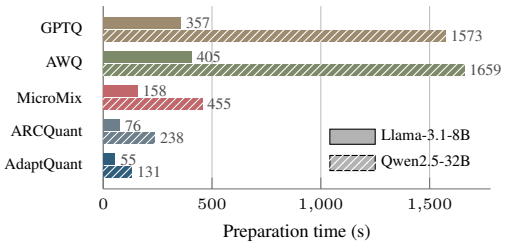


Figure 1: One-time cost of preparing a model for deployment on a single RTX 5090, from an unquantized checkpoint to a servable one.

Instead of incurring heavy calibration overhead when deploying large models, we propose AdaptQuant, a plug-and-play approach that calibrates only once and matches the accuracy of methods that recalibrate for every model. AdaptQuant selects an element format for each 16×32 tile from a single statistic computed in one pass over the tile. This tile size is not arbitrary: it matches exactly the operand granularity of Blackwell's block-scaled MMA instructions, so format selection aligns naturally with the hardware's native execution unit. The instruction, however, accepts only one element format per operand, so a tensor whose tiles differ in width cannot be fed to it as stored; AdaptQuant resolves this with a byte-level cast performed once per layer, leaving the GEMM itself the stock block-scaled kernel rather than a purpose-built one.

Our contributions are as follows:

- We derive a single per-tile statistic, computable in one pass over the tile, and identify the mechanism by which this statistic predicts the distortion a tile suffers under FP4. Thresholding this statistic yields a closed-form rule that assigns each 16×32 tile to MXFP4 or MXFP6, with MXFP8 admitted by the rule but never selected; the rule is derived once, offline, and reused across every model.

- We design GPU kernels around the block-scaled MMA's constraint of one element type per operand: weights are widened from their stored mixed pack once per layer, and activations are quantized directly into a uniform container, so a *stock* CUTLASS collective runs the GEMM with no per-format branching and no reordering kernel.

- Across 13 open-weight LLMs on a single RTX 5090, AdaptQuant attains the lowest mean perplexity increase over BF16 of any method evaluated, while spending fewer effective weight bits on average than MicroMix and storing smaller weights on 10 of the 13 models. AdaptQuant matches MicroMix's downstream accuracy, quantizes $2.9\times$ faster, and reaches a geomean $1.81\times$ BF16 prefill throughput.

2 Related Work

Microscaling formats. Microscaling formats share a single, wider-range scale across a block of low-bit elements, restoring dynamic range that no individual element could span on its own (Cococcioni et al., 2025). The Open Compute Project's MX specification (Rouhani et al., 2023) standardizes this idea across vendors: a 32-element block under a power-of-two E8M0 scale, with 8-, 6-, and 4-bit element encodings. NVIDIA's NVFP4 (Alvarez et al., 2025) instead pairs a finer 16-element block with an E4M3 scale, achieving higher fidelity at the same bit width, at the cost of being confined exclusively to NVIDIA hardware.

Mixed-precision quantization. Mixed-precision methods allocate higher precision to a small subset of sensitive values while aggressively quantizing the rest, a strategy originally introduced for integer formats (Zhao et al., 2024; Ashkboos et al., 2024). Closest to our work, MicroMix (Liu et al., 2026) brings this strategy to Microscaling formats on Blackwell, assigning each activation channel to MXFP4, MXFP6, or MXFP8 using calibration-derived thresholds together with an explicit channel reordering.

Table 1: Element data types used in this work, from the OCP MX standard. SQNR is measured under uniform-format quantization at $k = 32$ with an E8M0 scale.

MX format	Element f	m_f	V_f	u_f	Eff. bits b_f	SQNR (dB)
MXFP4	E2M1	1	6.0	0.5	4.25	17.33
MXFP6	E2M3	3	7.5	0.125	6.25	29.31
MXFP8	E4M3	3	448	2^{-9}	8.25	29.39

3 PRELIMINARIES

3.1 SIGNAL-TO-QUANTIZATION-NOISE RATIO

Signal-to-quantization-noise ratio (SQNR) is a standard fidelity metric in signal processing, where it quantifies the error introduced by representing a continuous or high-precision signal at reduced precision, such as in analog-to-digital conversion. We repurpose this metric to evaluate the error introduced by tensor quantization: let $\boldsymbol{x} \in \mathbb{R}^N$ denote a tensor (or tile) of unquantized values and $\hat{\boldsymbol{x}} \in \mathbb{R}^N$ its dequantized reconstruction. We measure reconstruction fidelity by the signal-to-quantization-noise ratio, the ratio of signal power to quantization-noise power in decibels:

$$\mathrm{SQNR}(\boldsymbol{x}, \hat{\boldsymbol{x}}) \triangleq 10 \log_{10} \frac{P_{\text{signal}}}{P_{\text{noise}}} = 10 \log_{10} \frac{\sum_{i=1}^{N} x_i^2}{\sum_{i=1}^{N} (x_i - \hat{x}_i)^2} \quad [\mathrm{dB}], \tag{1}$$

where $P_{\text{signal}} \triangleq \|\boldsymbol{x}\|_2^2$ and $P_{\text{noise}} \triangleq \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_2^2$. SQNR normalizes the squared error by the signal it corrupts, so tiles and layers of different magnitude are directly comparable — unnormalized error would let whichever tensor carries the largest values dominate any pooled quantity. The logarithmic scale is what makes the measure usable as a budget: a fixed number of decibels is a fixed fraction of a tensor's energy wherever it is spent, so one slack term applies uniformly across layers.

Bits and decibels. An extra mantissa bit halves the spacing of a format's grid, halving the rounding error and quartering rounding-error power, so one bit of precision is worth $10 \log_{10} 4 \approx 6.02$ dB of fidelity: bits and decibels measure the same thing in different units, which places rate and distortion on one scale. One consequence matters later: FP6 and FP8 share $m_f = 3$, so they are indistinguishable in average precision, with FP8 contributing only wider range — useful only for blocks that would clamp or flush under FP6.

3.2 MICROSCALING QUANTIZATION

An MX-compliant format pairs an element data type with a scale data type at block size k: a block of k elements $p_1, \ldots, p_k$ shares one scale s, decoding to $s\, p_i$. All concrete MX formats fix $k = 32$ and an E8M0 scale — an unsigned, exponent-only encoding using $s_{\text{bits}} = 8$ scale bits that constrains the shared scale to a power of two, so applying the scale is an exponent adjustment rather than a multiplication. An element format f has mantissa width m_f and largest/smallest representable magnitudes V_f, u_f (Table 1); $f \in \{\mathrm{FP4}, \mathrm{FP6}, \mathrm{FP8}\}$, reserving MXFP4/MXFP6/MXFP8 for the complete formats.

Quantization procedure. The specification's reference conversion of a block of values $x_1, \ldots, x_k$ to format f proceeds in two steps. First, the shared scale is set to the largest power of two not exceeding the block's maximum absolute value, divided by the largest power of two representable in the element type,

$$s = 2^{\sigma}, \qquad \sigma = \left\lfloor \log_2 \max_i |x_i| \right\rfloor - \lfloor \log_2 V_f \rfloor, \tag{2}$$

so that the block maximum maps near the top of the element format's range. Second, each element x_i is divided by the scale and rounded to nearest, with ties to even; a scaled value exceeding V_f is clamped to V_f, preserving sign, and one whose magnitude falls below $u_f/2$ rounds to zero. $Q_f(\cdot)$ denotes the resulting round-trip quantizer.

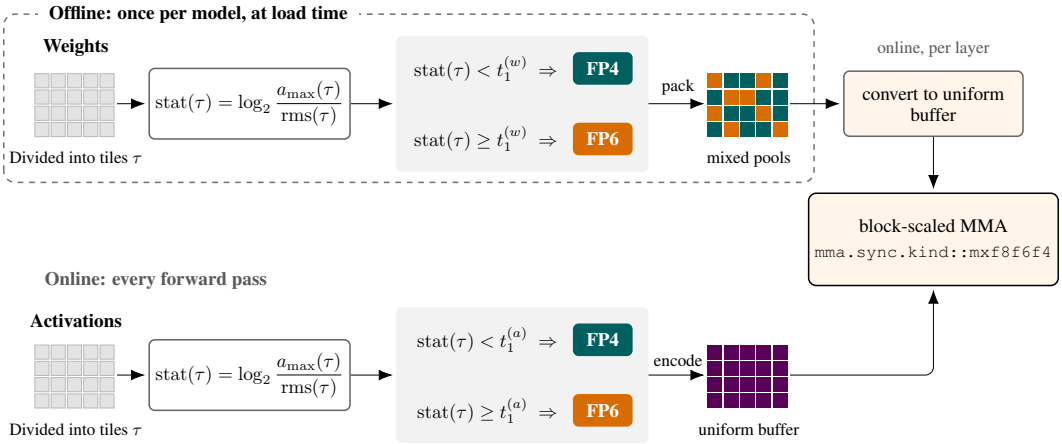


Figure 2: Per-tile format selection. Each 16×32 tile's statistic $\text{stat}(\tau)$ is thresholded to choose an element format. That selection rule allows a third format through t_2, but the derived thresholds switch it off on both operands, so only FP4 and FP6 appear here.

Effective bit width. Write $\text{bits}(f)$ for the payload width of element format f (4, 6, or 8). Amortizing the shared scale over the k elements of a block, the effective per-element cost is

$$b_f = \text{bits}(f) + \frac{s_{\text{bits}}}{k} = \text{bits}(f) + 0.25, \tag{3}$$

as listed in Table 1. By the same accounting, NVFP4 amortizes its E4M3 scale over $k = 16$ elements, for $4 + 8/16 = 4.5$ effective bits.

4 Method

4.1 Blackwell Block-Scaled MMA

Realizing the speedup promised by low-precision storage requires hardware that executes low-precision formats natively, rather than emulating them in software. We therefore build on consumer Blackwell GPUs, which expose native MX execution through a single instruction, `mma.sync.m16n8k32.kind::mxf8f6f4.block_scale`; critically, this instruction also fixes the granularity at which our method must perform format selection. The instruction treats the element format as a static, per-*operand* qualifier, and reads its two operands at different native shapes: $\boldsymbol{A}$ as a 16×32 ($M \times K$) region, $\boldsymbol{B}$ as a finer 8×32 ($N \times K$) one, each in a statically chosen element format. In a linear layer $\boldsymbol{A}$ carries the activations and $\boldsymbol{B}$ the weights. Because the format qualifier is fixed per operand, a scheme built on the stock kernel cannot vary the format inside a 16×32 region of $\boldsymbol{A}$. AdaptQuant therefore adopts this hardware granularity for both operands: a 16×32 tile corresponds to exactly one native region of $\boldsymbol{A}$ and two consecutive native regions of $\boldsymbol{B}$, so a single decision covers both. Throughout, we use *block* to denote a 1×32 region sharing one MX scale factor, and *tile* to denote a 16×32 region composed of 16 blocks, the granularity at which AdaptQuant selects one element format.

4.2 Per-Tile Format Selection

Given the per-tile granularity, we must decide which element format to assign to each tile, without recalibrating for each model and without first quantizing the tile. We propose a mixed-precision quantization method that makes this decision using only a statistic computed from the tile itself. Figure 2 illustrates this rule applied along both the weight and activation paths: the weight path runs once, offline, while the activation path runs on every forward pass.

Selection statistic. To decide a tile's format, our method computes a single scalar – cheap to derive from the tile itself – the log-ratio of the tile's largest magnitude to its root-mean-square:

$$a_{\max}(\tau) = \max_{i \in \tau} |x_i|, \qquad \mathrm{rms}(\tau) = \sqrt{\frac{1}{|\tau|} \sum_{i \in \tau} x_i^2}, \qquad \mathrm{stat}(\tau) \triangleq \log_2 \frac{a_{\max}(\tau)}{\mathrm{rms}(\tau)}. \tag{4}$$

One pass over the tile's values gives $\mathrm{stat}(\tau)$, with no per-model adjustment. Two thresholds $t_1 \le t_2$ on that scalar, derived once offline, replace calibration with a simple comparison rule:

$$f(\tau) \;=\; \begin{cases} \mathrm{FP4}, & \mathrm{stat}(\tau) < t_1, \\ \mathrm{FP6}, & t_1 \le \mathrm{stat}(\tau) < t_2, \\ \mathrm{FP8}, & \mathrm{stat}(\tau) \ge t_2. \end{cases} \tag{5}$$

Why $\mathrm{stat}(\tau)$ predicts distortion. Two properties of the MX format make $\mathrm{stat}(\tau)$ the right quantity to measure: a single scale per block, so only a value's size relative to the block maximum matters; and a lower bound on the element format's range, below which a value is lost entirely.

Every value in a block is divided by the same scale, which is set by the block's largest value (Eq. 2). Dividing by the scale leaves only the ratio between a value and the block maximum, not the value's raw magnitude:

$$z_i \;=\; \frac{|x_i|}{a_{\max}(\tau)} \;\in\; (0, 1], \tag{6}$$

with the largest value at $z = 1$ and every other value below it.

Small values are not inherently a problem, since a floating-point format resolves any value to a fixed fraction of the value's own magnitude. The problem is running off the bottom of the range: anything under half the format's smallest representable magnitude rounds to zero, and a zeroed value contributes its entire magnitude as error. In the normalized coordinate of Eq. 6, this cutoff sits at $z = 1/24$ for FP4, $z = 1/120$ for FP6, and $z \approx 1/460{,}000$ for FP8. Appendix A derives these cutoffs from the element grid.

Promoting a tile from FP4 to FP6 therefore brings two gains. One is a precision gain: about 12 dB by the rule of Section 3.1, from the two additional mantissa bits – a gain that applies equally to every tile, and so cannot help distinguish between them. The other is a range gain, from $1/24$ to $1/120$: this gain is large where many values sit below the cutoff, and zero where none do. Only the range gain separates the tiles worth promoting, and stat tracks it directly: since $\mathrm{rms}(z) = \mathrm{rms}(\tau)/a_{\max}(\tau)$, rewriting Eq. 4 in relative terms gives

$$\mathrm{stat}(\tau) \;=\; -\log_2 \mathrm{rms}(z), \tag{7}$$

the number of times the tile maximum must be halved to reach a typical value. A tile of uniform values has stat near zero and loses almost nothing to FP4; a tile with a few large values over a long tail has large stat, and its tail falls below the cutoff. Thresholding $\mathrm{stat}(\tau)$ as in Eq. 5 – computed from just one maximum and one sum of squares per tile, without ever running Q_{FP4} – therefore promotes exactly the tiles that stand to lose the most. Appendix A derives the cutoffs and attributes the measured distortion across the rounding, clamping, and range-loss regimes, showing that stat tracks the range-loss term specifically.

4.3 Threshold Derivation

To choose t_1 and t_2 in the rule of Eq. 5, we derive an algorithm that selects the pair minimizing the effective bits per value while keeping distortion below a target level. The derivation proceeds in three steps: it characterizes what each candidate format costs a given tile, poses the threshold choice as a rate–distortion program constrained to match NVFP4, and solves this program exactly over a grid of candidate thresholds.

Per-tile distortion. $\mathrm{stat}(\tau)$ orders tiles but never says how much a format actually distorts a given tile; the derivation measures the distortion directly, quantizing each tile once in each candidate format. The error is measured at the layer's output: a tile's 32 columns are input channels, and each element's squared error is weighted by w_c. This gives

$$e(\tau, f) \;=\; \sum_{i \in \tau} w_{c(i)} \big(x_i - Q_f(x_i)\big)^2, \qquad E_\tau \;=\; \sum_{i \in \tau} w_{c(i)}\, x_i^2, \tag{8}$$

where Q_f is the deployed quantizer forced into format f and $c(i)$ is the column of element i. The ratio $e(\tau, f)/E_\tau$ is the $P_{\text{noise}}/P_{\text{signal}}$ of Eq. 1 evaluated on a single tile, giving

$$\text{SQNR}(\tau, f) \;=\; -10 \log_{10} \frac{e(\tau, f)}{E_\tau} \tag{9}$$

as the output distortion of tile τ under format f. We weight the error by w_c rather than treating every channel equally, because channel c reaches the output only in proportion to w_c; an unweighted, operand-level distortion metric would let the thresholds be set by errors the output never actually sees. Likewise, we compute distortion per tile rather than tensor-wide: summing the noise and signal over all tiles before dividing would let a few loud tiles dominate the ratio and set the thresholds for the rest.

Problem formulation. Our method seeks the fewest possible bits without exceeding the distortion of a public standard: for each projection type, it minimizes the mean effective bits across a projection type's tiles, subject to the constraint that each layer's distortion not exceed what the standard would produce on the same tiles. We choose NVFP4 as this standard, a public format with strong accuracy. With b_f the effective bit width of format f, p the set of layers within a given projection type, and N_p the total number of tiles those layers contain, the derivation solves, for each operand and each p,

$$\min_{t_1^p \le t_2^p} \frac{1}{N_p} \sum_{\ell \in p} \sum_{\tau \in \ell} b_{f(\tau)} \quad \text{s.t.} \quad \sum_{\tau \in \ell} \frac{e(\tau, f(\tau))}{E_\tau} \;\le\; \sum_{\tau \in \ell} \frac{e_{\text{nv}}(\tau)}{E_\tau} \quad \text{for all } \ell \in p. \tag{10}$$

Here $e_{\text{nv}}(\tau)$ is the NVFP4 error of tile τ, measured under the same w_c. Although the sums run over tiles, the only variables are t_1^p and t_2^p: once the pair is fixed, every tile's format is determined. Each operand and each projection type therefore has its own program, and its own pair.

Candidate enumeration. We solve Eq. 10 efficiently using prefix sums over stat. Candidate thresholds are drawn from the grid $G = \{0, 0.02, \ldots, 4.50\} \cup \{\text{CAP}\}$: 4.5 is the largest value stat can take on a 16×32 tile, and CAP is a sentinel value set above it, so that no tile ever reaches it; setting a threshold to CAP therefore means "never promote." Measuring each tile once in every format reduces every tensor to arrays indexed by the grid, where each position stores the tile count and the normalized errors of every format and of NVFP4. A threshold pair splits the grid into three contiguous ranges, one per format, so computing the bit rate and each layer's error reduces to a range query over the grid-indexed arrays, answered with one prefix-sum array over the tile counts and one per layer over its per-format errors. The search enumerates every pair $t_1^p \le t_2^p$ on the grid, discards any pair that violates a layer's constraint, and returns the cheapest surviving pair, breaking ties toward more MXFP4. The chosen (t_1^p, t_2^p) is therefore the exact optimum on the grid. On every projection type of both operands, this optimum sets $t_2^p = \text{CAP}$: FP8 adds only range over FP6 (Section 3.1), too rarely needed to repay two extra bits, so deployed AdaptQuant uses FP4 and FP6 alone.

The derivation runs once, offline, on a single reference model; the resulting thresholds are then applied unchanged to every model we evaluate, so quantizing a new model requires no data at all. Algorithm 1 collects these steps, and Appendix B provides the supporting details.

4.4 GPU Kernel Implementation

Realizing AdaptQuant's per-tile format assignment as an actual speedup on hardware requires a dedicated GPU kernel. Section 4.2 established that different tiles within one operand may be stored in different formats, but the MMA instruction of Section 4.1 admits only one static format per operand: the whole operand must therefore be made uniform again immediately before the GEMM is launched. This raises two problems that stock kernels do not face: how to store an operand whose tiles have different bit widths, and how to convert it into a single format cheaply enough that the conversion does not erase the gains from mixed precision. AdaptQuant addresses both with two techniques.

Storage. A mixed tensor has no single stride, since its tiles are stored at different bit widths. AdaptQuant therefore gives each element format its own contiguous pool and maintains a per-tile index recording which pool holds each tile and at what offset within it. FP4 tiles are packed at four

Table 2: WikiText-2 and C4 perplexity (↓) on six models.

	WikiText-2					C4				
Model	BF16	MXFP4	NVFP4	MicroMix	Ours	BF16	MXFP4	NVFP4	MicroMix	Ours
Llama-2-7B	5.47	7.02	5.76	**5.69**	5.79	6.97	8.69	7.29	**7.24**	7.31
Llama-3.1-8B	6.24	8.31	6.95	**6.77**	6.84	8.96	11.66	9.93	**9.78**	9.80
Qwen2.5-7B	6.85	8.67	7.29	7.18	**7.14**	10.45	12.54	10.92	10.85	**10.76**
Qwen2.5-32B	5.02	6.00	5.41	5.56	**5.31**	8.95	9.78	9.20	9.37	**9.14**
Qwen3-8B	9.73	11.86	10.05	10.10	**10.00**	13.30	15.84	13.77	13.77	**13.75**
Mistral-7B-v0.1	5.25	6.57	5.48	**5.43**	**5.43**	7.74	9.30	8.02	7.98	**7.96**

bits per element into one pool and FP6 tiles at six bits per element into another; only these two pools exist, since the derived thresholds never select FP8.

Conversion. Immediately before each layer's GEMM, AdaptQuant converts the layer's mixed tiles once into a uniform operand, writing into a small buffer shared by the whole model. The conversion must happen outside the GEMM rather than inside it: operand movement within the GEMM is already handled by the tensor-memory accelerator at no warp cost, whereas converting formats element-by-element inside the inner loop would require touching every byte in software and paying the conversion cost once per main-loop iteration rather than once per layer. The conversion instead runs as a single pass before the GEMM, on a side stream that overlaps the previous layer's GEMM. Appendix C gives the operand layouts and the scheduling of the conversion.

5 Experiments

5.1 Experimental Setup

Models. Evaluation covers 13 open-weight LLMs spanning Llama-2/3 (Touvron et al., 2023; Grattafiori et al., 2024), Qwen2.5/3 (Qwen et al., 2025; Yang et al., 2025), and Mistral (Jiang et al., 2023), from 3B to 32B. For readability, the main text reports six representative models spanning all three families and both small and large scale; full results on all 13 models are given in Appendix D.

Baselines. The comparison includes BF16 (unquantized), uniform MXFP4, uniform NVFP4, and MicroMix (Liu et al., 2026), the closest prior mixed-precision MX-format method and the strongest baseline here.

Metrics. Perplexity is reported on WikiText-2 (Merity et al., 2017) and C4 (Dodge et al., 2021) (sequence length 2048); zero-shot accuracy on PIQA (Bisk et al., 2020), HellaSwag (Zellers et al., 2019), WinoGrande (Sakaguchi et al., 2021), ARC-e and ARC-c (Clark et al., 2018), LAMBADA (Paperno et al., 2016), and BoolQ (Clark et al., 2019) via LM-Eval (Gao et al., 2024), averaged across tasks; and 5-shot MMLU (Hendrycks et al., 2021).

Hardware. All measurements run on a single NVIDIA GeForce RTX 5090 (32 GB, compute capability 12.0/`sm_120`) with driver 595.84 and CUDA 13.2.

5.2 Perplexity

Table 2 reports WikiText-2 and C4 perplexity for our method and four baselines on six models spanning all three families; the means below are taken over all 13 models (Table 5). AdaptQuant attains the lowest mean increase over BF16 of any quantized method on both datasets, 5.0% on WikiText-2 and 3.9% on C4, against $5.6\%/4.5\%$ for MicroMix and $6.1\%/4.9\%$ for NVFP4. The margin is largest where the model is largest: on Qwen2.5-32B and Qwen3-32B, neither of which the RTX 5090 can serve in BF16, AdaptQuant beats MicroMix by 0.25 and 0.29 perplexity on WikiText-2 and by 0.23 on C4 for both. No baseline beats AdaptQuant by more than 0.10 perplexity on any of the 13 models; the largest gap is on Llama-2-7B (5.79 vs. MicroMix's 5.69).

Table 3: Zero-shot accuracy (mean over 7 tasks) and 5-shot MMLU (↑) on six models.

	Zero-shot Avg.					5-shot MMLU				
Model	BF16	MXFP4	NVFP4	MicroMix	Ours	BF16	MXFP4	NVFP4	MicroMix	Ours
Llama-2-7B	70.70	66.32	69.32	**69.95**	69.86	45.82	33.57	42.82	**43.66**	41.84
Llama-3.1-8B	75.42	67.78	73.29	**73.81**	73.18	65.35	54.73	62.28	**62.83**	62.24
Qwen2.5-7B	73.63	68.78	72.53	72.57	**73.67**	74.16	66.54	72.18	72.57	**72.72**
Qwen2.5-32B	77.00	75.70	76.85	76.60	**76.95**	83.21	80.32	82.55	81.63	**82.72**
Qwen3-8B	72.47	67.31	71.21	**71.62**	71.43	74.89	68.25	72.75	73.11	**73.60**
Mistral-7B-v0.1	75.66	71.42	74.63	74.76	**74.79**	62.55	50.71	60.01	60.65	**60.89**

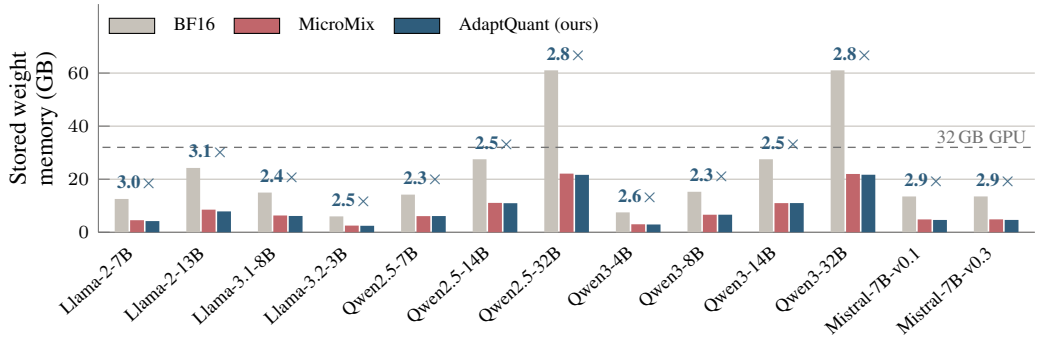


Figure 3: Stored weight memory; labels give AdaptQuant's reduction relative to BF16. Qwen2.5-32B and Qwen3-32B exceed the card's 32 GB in BF16 weights alone, so neither is servable unquantized on this GPU.

5.3 Downstream Task Accuracy

Table 3 reports zero-shot accuracy averaged over seven tasks and 5-shot MMLU for our method and four baselines on the same six models; retention figures below are averaged over all 13 models (Table 6). AdaptQuant and MicroMix each retain 98.6% of BF16 zero-shot accuracy; on MMLU MicroMix leads, 97.1% against 96.9%. Uniform MXFP4 retains only 93.9% and 87.4%. Neither mixed-precision method leads by much on any of the 13 models: AdaptQuant's largest zero-shot gain over MicroMix is 1.10 points on Qwen2.5-7B (73.67 vs. 72.57) and its largest MMLU gain 1.09 on Qwen2.5-32B (82.72 vs. 81.63), while its largest deficit is 1.82 MMLU points on Llama-2-7B (41.84 vs. 43.66).

5.4 Memory Footprint and Quantization Cost

Figure 3 reports stored weight memory for all 13 models. Our method reduces it by 2.31 to 3.08× against BF16, a geomean of 2.65× where MicroMix reaches 2.58×, spending 5.17 effective weight bits per parameter on average, per-tile index included, against MicroMix's 5.33. AdaptQuant stores the smaller model on 10 of the 13; on the other three the difference is at most 0.03 GB (Qwen2.5-7B, 6.11 vs. 6.08).

Quantization time separates the two more sharply. Figure 4 shows our method faster on every model, by 2.18× at the narrowest (Qwen3-4B, 61.8 s vs. 135.0 s) and 3.47× at the widest (Qwen2.5-32B, 131.4 s vs. 455.4 s), for a mean of 72.3 s against 211.6 s. Appendix D.2 gives the per-model numbers behind both plots.

5.5 Inference Throughput

Figure 5 reports prefill throughput at sequence length 2048. Our method reaches a geomean 1.81× BF16 over the 11 models BF16 can serve, ahead of MicroMix on 7 of them. The gains are largest on the two smallest models, 1.67× against 1.54× on Llama-3.2-3B and 1.56× against 1.44× on

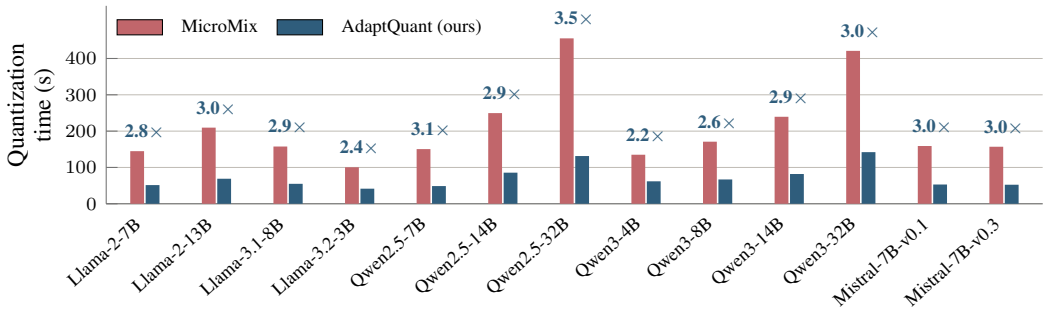


Figure 4: One-time quantization cost, all 13 models (lower is better). Labels give AdaptQuant's speedup over MicroMix.

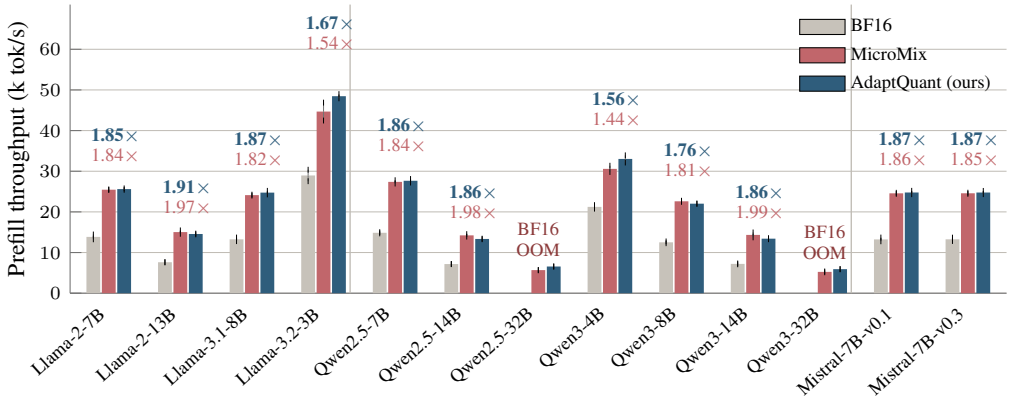


Figure 5: Prefill throughput at sequence length 2048, averaged over the batch sizes in $\{1, 4, 8, 16, 32\}$ completed by every method shown; error bars give the standard deviation across those batch sizes. Labels give each method's speedup over BF16.

Qwen3-4B, and where MicroMix is ahead the gap is never more than 0.13 (Qwen3-14B). On Qwen2.5-32B and Qwen3-32B, which BF16 cannot serve at all, AdaptQuant delivers $1.16\times$ and $1.13\times$ MicroMix's throughput, and no model or batch size measured is slower than BF16, the minimum speedup being $1.53\times$. Appendix D.3 gives the per-configuration numbers.

6 Conclusion

In this paper we propose AdaptQuant, a mixed-precision MX quantization method that avoids per-model calibration while matching the accuracy of the strongest mixed-precision baseline at fewer effective weight bits. Format selection is a single statistic computed in one pass over each 16×32 tile, matching the operand shape Blackwell's block-scaled MMA reads, thresholded by one scalar per operand and projection type, derived once, offline, against NVFP4 and applied unchanged to every model. Quantizing a new model is therefore one pass over the weights, $2.9\times$ faster than the strongest mixed-precision baseline. Across 13 open-weight LLMs our method attains the lowest mean perplexity increase over BF16 of any method compared, 5.0% on WikiText-2 and 3.9% on C4, while storing the smaller model on 10 of the 13. Widening every operand to a uniform format before the GEMM lets a stock block-scaled collective reach a geomean $1.81\times$ BF16 prefill throughput, never slower than BF16 on any measured shape, and serve two 32B models BF16 cannot load at all.

AI use statement

In this work, we used generative AI tools for formulating mathematical claims, proving mathematical claims, developing theoretical models, and implementing methods. We have not used generative

AI tools for proposing or refining hypotheses, designing research methodology or experiments, or interpreting results, and generating synthetic data sets, translation, data cleaning, and qualitative data analysis are not applicable to this work. Additionally, we used generative AI tools for editing software code, drafting and polishing the paper, identifying related literature, and proposing a title. We have reviewed all AI-assisted work. The core ideas of the method were conceived by the authors, and all AI-assisted proofs, code, and text were verified by the authors. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

REFERENCES

Eduardo Alvarez, Omri Almog, Eric Chung, Simon Layton, Dusan Stosic, Ronny Krashinsky, and Kyle Aubrey. Introducing NVFP4 for efficient and accurate low-precision inference. NVIDIA Technical Blog, June 2025. URL https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/.

Saleh Ashkboos, Ilia Markov, Elias Frantar, Tingxuan Zhong, Xincheng Wang, Jie Ren, Torsten Hoefler, and Dan Alistarh. QUIK: Towards end-to-end 4-bit inference on generative large language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 3355–3371, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.197. URL https://aclanthology.org/2024.emnlp-main.197/.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. In *Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1300. URL https://aclanthology.org/N19-1300/.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018. URL https://arxiv.org/abs/1803.05457.

Marco Cococcioni, Dario Pagani, and Federico Rossi. Microscaling floating point formats for large language models, 2025. URL https://arxiv.org/abs/2510.01863.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022. URL https://arxiv.org/abs/2208.07339.

Jesse Dodge, Maarten Sap, Ana Marasović, William Agnew, Gabriel Ilharco, Dirk Groeneveld, Margaret Mitchell, and Matt Gardner. Documenting large webtext corpora: A case study on the colossal clean crawled corpus. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 1286–1305, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.98. URL https://aclanthology.org/2021.emnlp-main.98/.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2023. URL https://arxiv.org/abs/2210.17323.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL https://zenodo.org/records/12608602.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023. URL https://arxiv.org/abs/2310.06825.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. In *MLSys*, 2024.

Wenyuan Liu, Haoqian Meng, Yilun Luo, Peng Zhang, and Xindian Ma. Micromix: Efficient mixed-precision quantization with microscaling formats for large language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=P5OKoZdwlB.

Haoqian Meng, Yilun Luo, Yafei Zhao, Wenyuan Liu, Peng Zhang, and Xindian Ma. Arcquant: Boosting nvfp4 quantization with augmented residual channels for llms. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8609–8623. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.388. URL http://dx.doi.org/10.18653/v1/2026.acl-long.388.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017. URL https://openreview.net/forum?id=Byj72udxe.

NVIDIA Corporation. NVIDIA RTX Blackwell GPU Architecture, 2025. URL https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf. Version 1.1.

Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In Katrin Erk and Noah A. Smith (eds.), *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1525–1534, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1144. URL https://aclanthology.org/P16-1144/.

Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL https://arxiv.org/abs/2412.15115.

Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, Stosic Dusan, Venmugil Elango, Maximilian Golub, Alexander Heinecke, Phil James-Roxby, Dharmesh Jani, Gaurav Kolhe, Martin Langhammer, Ada Li, Levi Melnick, Maral Mesmakhosroshahi, Andres Rodriguez, Michael Schulte, Rasoul Shafipour, Lei Shao, Michael Siu, Pradeep Dubey, Paulius Micikevicius, Maxim Naumov, Colin Verrilli, Ralph Wittig, Doug Burger, and Eric Chung. Microscaling data formats for deep learning, 2023. URL https://arxiv.org/abs/2310.10537.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: an adversarial winograd schema challenge at scale. *Commun. ACM*, 64(9):99–106, August 2021. ISSN 0001-0782. doi: 10.1145/3474381. URL https://doi.org/10.1145/3474381.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, et al. Llama 2: Open foundation and fine-tuned chat models, 2023. URL https://arxiv.org/abs/2307.09288.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In Anna Korhonen, David Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1472. URL https://aclanthology.org/P19-1472/.

Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. Atom: Low-bit quantization for efficient and accurate llm serving. In P. Gibbons, G. Pekhimenko, and C. De Sa (eds.), *Proceedings of Machine Learning and Systems*, volume 6, pp. 196–209, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/file/5edb57c05c81d04beb716ef1d542fe9e-Paper-Conference.pdf.

A Error Attribution

Section 4.2 argues that FP4 distortion splits into a part fixed by the format and a part fixed by the tile, and that stat tracks the second. This appendix states the three regimes that split refers to and measures how the distortion actually divides between them.

A.1 Why relative error is flat, and where the range ends

Section 4.2 asserts two things about a floating-point element format: that its relative error is the same throughout its representable range, and that the range has a sharp bottom at $z = u_f/2V_f$. Both follow from the grid.

Flat inside the range. Between consecutive powers of two 2^e and 2^{e+1} a format with m_f mantissa bits has 2^{m_f} evenly spaced values, so the gap is $\Delta_e = 2^{e-m_f}$ and round-to-nearest costs at most $\Delta_e/2$. Any value in that interval is at least 2^e, so the *relative* error is at most

$$\frac{\Delta_e/2}{2^e} \;=\; 2^{-m_f-1}, \tag{11}$$

independent of e and therefore of the value. Squaring, noise power scales as 2^{-2m_f}: each mantissa bit quarters it, which is $10\log_{10} 4 \approx 6.02$ dB. Two formats with the same m_f are equally precise here, which is why E2M3 and E4M3 are indistinguishable in Table 1 despite E4M3's much wider exponent range — exponent bits change which values are reachable, not how well reachable values are resolved.

Where the range ends. The smallest magnitude the format represents is u_f, and nothing lies between 0 and u_f, so round-to-nearest sends anything under $u_f/2$ to zero. The shared scale places the block maximum near the format's largest magnitude V_f (Eq. 2), so in the normalized coordinate of Eq. 6 the cutoff sits at

$$z < \frac{u_f}{2V_f} \iff \frac{a_{\max}}{|x_i|} > \frac{2V_f}{u_f}, \tag{12}$$

which evaluates to $24\times$ for E2M1, $120\times$ for E2M3 and $458{,}752 \approx 4.6 \times 10^5$ for E4M3. The factor of two beyond the format's raw span V_f/u_f is round-to-nearest: a value is already sent to zero at the midpoint $u_f/2$, not only once it falls to u_f.

Clamping. Symmetrically, an E8M0 scale can place the block maximum only at power-of-two granularity. Eq. 2 therefore leaves the scaled maximum somewhere in $[2^{e_V}, 2^{e_V+1})$ with $e_V = \lfloor \log_2 V_f \rfloor$ — for E2M1, in $[4, 8)$ against $V_f = 6$ — so whenever the scaled maximum lands in the upper part of that interval, the largest few elements of a block exceed V_f and are capped. Clamping depends on where the maximum falls within its binade — the interval between two consecutive powers of two — rather than on the block's shape, so clamping does not vary systematically with stat.

A.2 Which term varies, measured

The decomposition says the rounding contribution is uniformly bounded while the low-magnitude loss is not. It does not say how the total distortion divides between them in practice, or — what selection actually depends on — which part stat tracks. We measure both by quantizing tiles of Llama-3.2-3B to FP4 with the deployed quantizer and attributing each element's squared error to the regime the element fell in. Write D_{FP4} for the resulting total squared error, so that each regime's share of D_{FP4} is the fraction of FP4's damage it accounts for.

Table 4: FP4 error attribution, Llama-3.2-3B: $393{,}216$ weight tiles (all projection types) and $59{,}904$ activation tiles (WikiText-2, layers 1, 14, 27). ρ is the Spearman rank correlation between a regime's per-tile error and stat. On both operands the dominant term is flat in stat and the term stat tracks is the low-magnitude loss; on activations that term is more than twice as large a share.

	weights		activations	
regime	share of D_{FP4}	ρ vs. stat	share of D_{FP4}	ρ vs. stat
low-magnitude (subnormal + flush)	11.9%	$+0.59$	29.0%	$+0.81$
normal-range rounding	74.4%	-0.08	50.4%	-0.29
clamping	13.7%	$+0.16$	20.7%	$+0.30$
total	100%	$+0.31$	100%	$+0.73$

Table 4 reports the result. Three observations hold on both operands. First, normal-range rounding is the largest single contributor yet is uncorrelated (weights) or anti-correlated (activations) with stat: it behaves as the flat component the rounding regime of Appendix A.1 describes, and no format decision reduces it other than by paying for mantissa bits everywhere. Second, the low-magnitude loss is the term that varies, and stat tracks it strongly. Third, clamping is sizeable but governed by where a block's maximum falls within its binade rather than by the block's shape.

The operands differ in degree, not in kind. On weights the two tile-independent terms together account for 88% of the distortion (74.4% rounding plus 13.7% clamping), so stat correlates only moderately with the *total* ($+0.31$) while still selecting well — ranking by the total would be ranking mostly by the part no choice can change. On activations the varying term is 29% and stat tracks even the total at $+0.73$. The statistic is thus most predictive on the operand where format choice has the most to recover, and its weaker correlation on weights reflects the size of the irreducible part rather than a failure of the ordering.

Algorithm 1 Threshold derivation. Every tile contributes $\mathrm{stat}(\tau)$ and its normalized errors $e(\tau, f)/E_\tau$, $e_{\mathrm{nv}}(\tau)/E_\tau$ to one grid position of H, after which the tensor is discarded. SOLVE(o, s) solves Eq. 10 exactly on the grid G for every projection type of operand o, with each layer's reference relaxed by a slack of s decibels. The body splits an optional target $b^\star$, the mean of the two operands' effective bits, holding activations at NVFP4 parity and letting the weights absorb the rest.

Require: reference model, text corpus, grid G, optional target bits $b^\star$
Ensure: a threshold pair (t_1^p, t_2^p) for each operand and projection type p

1: **for all** eligible linear layers ℓ, on their forward pre-hooks **do**
2: $\quad M[\ell]$ += $\frac{1}{T}\,\mathrm{diag}(X^\top X)$; add X's tiles to $H[\mathrm{act}, \ell]$ under $w_c = \|W_{:,c}\|^2$
3: **end for**
4: **for all** eligible linear layers ℓ **do** ▷ after the pass: this weighting needs M
5: $\quad$ add W_ℓ's tiles to $H[\mathrm{wt}, \ell]$ under $w_c = M[\ell]_c$
6: **end for**
7: **procedure** SOLVE(operand o, slack s)
8: $\quad$ **for all** projection types p **do**
9: $\quad\quad$ $\mathrm{bitrate}[\,\cdot\,] \leftarrow$ ENUMERATE$\big(\bigcup_{\ell \in p} H[o, \ell]\big)$ ▷ all layers of p together; prefix sums score every candidate at once
10: $\quad\quad$ $A \leftarrow \{(t_1^p, t_2^p) : \mathrm{err}_\ell \le 10^{s/10}\,\mathrm{ref}_\ell \;\; \forall \ell \in p\}$ ▷ one inequality per layer
11: $\quad\quad$ $(t_1^p, t_2^p) \leftarrow \arg\min_A \mathrm{bitrate}$, ties $\rightarrow$ more MXFP4
12: $\quad$ **end for**
13: $\quad$ **return** the pairs $\{(t_1^p, t_2^p)\}_p$ for operand o, and the bits they spend over all of o
14: **end procedure**
15: $b^{(a)} \leftarrow$ bits of SOLVE(act, 0); $b^{(w)} \leftarrow 2b^\star - b^{(a)}$ ▷ activations at NVFP4 parity
16: bisect $s \ge 0$ until SOLVE(wt, s) spends $b^{(w)}$ ▷ bit rate is monotone in s; roles swap if the weights would fall below all-FP4

B THRESHOLD DERIVATION SUPPLEMENTS

This appendix supplements Section 4.3 and Algorithm 1: when the derivation is run and on what, where the channel weights w_c come from, the prefix-sum algebra that scores the whole grid at once, and what the procedure costs.

The derivation is run once. Algorithm 1 is not part of quantizing a model. It is run a single time, offline, on one reference model (Llama-3.1-8B) over a single WikiText-2 sequence of 2048 tokens, and the thresholds it returns are applied unchanged to every model in Section 5. Quantizing a new model therefore needs no forward pass over text, no calibration set and no re-derivation: its weight formats follow from $\mathrm{stat}(\tau)$ on its own weights, and its activation formats from $\mathrm{stat}(\tau)$ on each tile as it enters the GEMM. This is what separates the corpus above from a per-model calibration set. The corpus is read once, to fix one threshold per operand and projection type, 14 numbers for two operands across seven projection types, rather than once per deployed model.

Channel weighting. The weights w_c of Eq. 8 come from propagating an operand error through the GEMM. Writing the layer as $Y = XW^\top$ with $X \in \mathbb{R}^{T\times C}$ and $W \in \mathbb{R}^{O\times C}$, so that column c of W holds the weights leaving input channel c, an error $\Delta x_{:,c}$ confined to that channel of the activation gives $\|\Delta Y\|_F^2 = \|\Delta x_{:,c}\|^2\,\|W_{:,c}\|^2$, and an error $\Delta W_{:,c}$ in the weight gives $\|\Delta Y\|_F^2 = \|\Delta W_{:,c}\|^2 \sum_t x_{tc}^2$. Each is exact for a single channel; summing them over channels is the approximation, and it holds when the per-channel errors are mutually uncorrelated.

The weight-side factor $\sum_t x_{tc}^2 = (X^\top X)_{cc}$ is the quantity GPTQ's layer-wise objective minimizes against (Frantar et al., 2023). It is the one place the derivation needs observed activations, and it is accumulated during the pass of line 1 rather than recomputed, so the activation operand is never held past the hook that saw it. The activation-side factor $\|W_{:,c}\|^2$ needs no data at all. Writing $\tilde{w}_c$ for whichever raw factor applies, the weight entering Eq. 8 is $w_c = \tilde{w}_c \big/ \big(\frac{1}{C}\sum_{c'} \tilde{w}_{c'}\big)$, rescaled to unit mean so that reported SQNR values stay on their usual scale. That rescaling does not change the search: $e(\tau, f)$ and E_τ are both linear in w_c, so any constant factor cancels from their ratio. The same cancellation makes the T in $\sum_t x_{tc}^2$ immaterial, which is why the moment can be accumulated

as a running mean over hook calls. What w_c does change is the weight of one channel against another, and so of one tile against another.

Prefix-sum algebra. Both sides of Eq. 10 reduce to the difference of two array entries, which is what lets the whole grid be scored at once. Write $G = (g_1 < \cdots < g_m)$ and place each tile at the position i with $g_i \le \mathrm{stat}(\tau) < g_{i+1}$, taking $g_{m+1} = \infty$; the position below $g_1 = 0$ is empty, since $a_{\max} \ge \mathrm{rms}$ makes $\mathrm{stat} \ge 0$. Line 1 accumulates at each position the tile count n_i and, for each layer ℓ and format f, the normalized errors

$$\varepsilon^{\ell}_{f,i} = \sum_{\tau \in \ell,\ \mathrm{pos}(\tau)=i} \frac{e(\tau, f)}{E_\tau}, \qquad \nu^{\ell}_{i} = \sum_{\tau \in \ell,\ \mathrm{pos}(\tau)=i} \frac{e_{\mathrm{nv}}(\tau)}{E_\tau}, \tag{13}$$

with exclusive prefix sum arrays N_i, $\mathcal{E}^{\ell}_{f,i}$, $\mathcal{N}^{\ell}_{i}$. Setting $t^p_1 = g_i$ and $t^p_2 = g_j$ with $i \le j$, Eq. 5 sends positions below i to FP4, $[i, j)$ to FP6 and j upward to FP8, each a contiguous range, so both sides of Eq. 10 are differences of two entries:

$$R(i,j) = \tfrac{1}{N_p}\Big[b_{\mathrm{FP4}} N_i + b_{\mathrm{FP6}}\big(N_j - N_i\big) + b_{\mathrm{FP8}}\big(N_{m+1} - N_j\big)\Big], \tag{14}$$

$$\mathrm{err}_\ell(i,j) = \mathcal{E}^{\ell}_{\mathrm{FP4},\,i} + \big(\mathcal{E}^{\ell}_{\mathrm{FP6},\,j} - \mathcal{E}^{\ell}_{\mathrm{FP6},\,i}\big) + \big(\mathcal{E}^{\ell}_{\mathrm{FP8},\,m+1} - \mathcal{E}^{\ell}_{\mathrm{FP8},\,j}\big), \tag{15}$$

with $\mathrm{ref}_\ell = \mathcal{N}^{\ell}_{m+1}$, feasible set $A(s) = \{(i,j) : i \le j,\ \mathrm{err}_\ell(i,j) \le 10^{s/10}\mathrm{ref}_\ell\ \forall \ell \in p\}$, and line 11 returning $(t^p_1, t^p_2) = (g_{i^\star}, g_{j^\star})$ for $(i^\star, j^\star) = \arg\min_{A(s)} R$. The counts N_i run over every layer of p together, since Eq. 14 is the bit rate the model pays; the errors are each layer's own, which is why the two are accumulated separately.

Both sides of the constraint are sums of per-tile noise-to-signal ratios over the same tiles, so dividing by the layer's tile count $|\ell|$ and taking $-10\log_{10}$ turns it into a statement about SQNR. Writing $\mathrm{SQNR}_\ell = -10\log_{10}\big(\mathrm{err}_\ell/|\ell|\big)$ and $\mathrm{SQNR}^{\mathrm{nv}}_\ell = -10\log_{10}\big(\mathrm{ref}_\ell/|\ell|\big)$,

$$\mathrm{err}_\ell(i,j)\ \le\ 10^{s/10}\,\mathrm{ref}_\ell \qquad \Longleftrightarrow \qquad \mathrm{SQNR}_\ell(i,j)\ \ge\ \mathrm{SQNR}^{\mathrm{nv}}_\ell - s \quad \mathrm{dB}. \tag{16}$$

This is what fixes the slack's units: s is the number of decibels of output-space SQNR each layer may give up against NVFP4, and $s = 0$ is parity. The search is carried out in the linear form, since the two feasible sets are identical under a monotone transform but only the linear form is a range sum. The dB form is what the search reports: the SQNR of the chosen assignment against that of the reference, and the tightest per-layer margin $\min_{\ell \in p}\big(\mathrm{SQNR}_\ell - \mathrm{SQNR}^{\mathrm{nv}}_\ell\big)$, which near zero means one layer is setting the thresholds for all of them.

Three properties follow. Each candidate costs $O(1)$ once the prefix sum arrays exist, so the optimum is exact over $G \times G$ rather than a relaxation. Feasible sets nest, $s \le s' \Rightarrow A(s) \subseteq A(s')$, so $R^\star(s) = \min_{A(s)} R$ is monotone non-increasing and the bisection of line 16, 48 iterations on $s \in [0, 60]$ dB, is well posed. And $t^p_2 = \textsc{cap}$ pins $j = m$, collapsing the $m(m{+}1)/2 = 25{,}878$ ordered pairs to the 227 single cuts, so at the deployed operating point the sweep is one-dimensional.

Ties on the bit rate are broken toward the pair with more MXFP4 tiles: at equal bits and equal admissibility the assignment leaning hardest on the cheapest format is the one that stays cheap if a format is ever retired, so the preference costs nothing and is taken. If $A(s)$ is empty, meaning no threshold pair keeps every layer at the reference, the search reports the minimum-error pair and flags that the reference was not met. No operating point reported here reached that fallback: every projection type of both operands admitted a feasible pair at parity.

Complexity. Measurement costs one forward pass over the corpus plus one pass over the weights, at a peak memory of one layer's operand. Everything after it, lines 9 to 16, runs on the position arrays alone, solving a projection type of L layers in $O(Lm)$ time and memory once t^p_2 is pinned, so the 48 re-solves of line 16 take seconds. Re-deriving at a different bit target reuses the cached arrays and repeats no pass over the model.

C GPU Kernel Implementation Details

This appendix gives the full mechanics behind Section 4.4: the stored operand layout and the conversion kernel's scheduling.

C.1 STORED OPERAND

A mixed-width tensor has no fixed stride, so codes are not stored in place. Each element format owns a contiguous pool — FP4 nibble-packed, FP6 as a packed 6-bit stream — and a per-tile table locates each tile inside its pool: a format tag and an element offset. Scales occupy a separate plane, laid out in the blocked order the tensor cores address them in.

Two properties of this layout are used throughout. A tile's 512 codes are *contiguous* within its pool, occupying 256 B at FP4 and 384 B at FP6; and pool offsets are multiples of the tile size, so the byte offsets derived from them ($\mathrm{offset}/2$ for FP4, $3\,\mathrm{offset}/4$ for FP6) are exact and aligned. The per-tile tables cost 5 B per 512 elements, or 0.078 bits/parameter; this is included in the stored memory of Table 7 but not in its W-bits column, which counts element codes and scales only.

C.2 CONVERSION KERNEL

The conversion assigns one thread to each group of 16 consecutive codes within a row. A group lies wholly inside one tile, so it has a single format tag, a single source pool, and one contiguous source run; and because both 16 codes at FP4 (8 B) and at FP6 (12 B) are whole numbers of bytes, the source and destination offsets of every group divide exactly. No shared memory, no cross-thread communication, and one dependent load per thread.

For an FP6 tile the group is copied unchanged: source and destination encode identically and only the position differs. For an FP4 tile each code is widened by a two-bit left shift. E2M1 and E2M3 share the sign bit, the two exponent bits and the exponent bias, so appending two zero mantissa bits yields the E2M3 code of exactly the same value, subnormals included. Both formats also share $\lfloor \log_2 V_f \rfloor = 2$, so the E8M0 scale computed for FP4 (Eq. 2) is the one FP6 would use, and the scale plane is left untouched. The 16 widened codes are re-packed into 96 bits, three output words from two input words. Measured across the projection shapes of an 8B-class model, the pass costs 2.4–3.0% of the GEMM it feeds and sustains an effective bandwidth close to the device limit, as expected of a kernel that reads once and writes once.

C.3 SCHEDULING THE CONVERSION

Converted operands are transient, so a small pool of buffers — each sized to the largest layer in the model, resident cost (number of buffers) $\times$ (largest layer), independent of depth — is shared across layers, acquired before a layer's GEMM and released after. Overlap needs to know which layer runs next: the pool records the request order from the first forward pass and, once that order repeats, converts ahead of time on a side stream; until then, conversion runs synchronously on the compute stream — correct, merely unoverlapped.

Two events per buffer carry the cross-stream ordering, one per edge: a *ready* event, recorded on the side stream when conversion completes and awaited by the compute stream before the GEMM, and a *done* event, recorded on the compute stream after the GEMM and awaited by the side stream before reuse. Prefetch depth is the one tunable; three buffers measured best, since two GEMMs of lead time cover a conversion where one does not, and a fourth buffer stops paying for itself.

Fused blocks follow the same rule: a gated MLP's elementwise product feeds the down-projection's quantization, so that projection's operand is acquired from the pool like any other layer rather than cached separately, which would forfeit the pool's saving for that one projection type.

D FULL BENCHMARK RESULTS

D.1 PERPLEXITY AND DOWNSTREAM ACCURACY

Table 5 and Table 6 give perplexity, zero-shot accuracy, and 5-shot MMLU for all 13 evaluated models.

D.2 MEMORY FOOTPRINT AND QUANTIZATION COST

Table 7 gives the per-model data behind Figures 4 and 3.

Table 5: WikiText-2 and C4 perplexity (↓), all 13 models.

	WikiText-2					C4				
Model	BF16	MXFP4	NVFP4	MicroMix	Ours	BF16	MXFP4	NVFP4	MicroMix	Ours
Llama-2-7B	5.47	7.02	5.76	**5.69**	5.79	6.97	8.69	7.29	**7.24**	7.31
Llama-2-13B	4.89	5.84	5.07	**5.05**	5.11	6.47	7.54	**6.66**	6.66	6.69
Llama-3.1-8B	6.24	8.31	6.95	**6.77**	6.84	8.96	11.66	9.93	**9.78**	9.80
Llama-3.2-3B	7.82	10.28	8.61	**8.36**	8.44	10.44	13.64	11.65	11.38	**11.37**
Qwen2.5-7B	6.85	8.67	7.29	7.18	**7.14**	10.45	12.54	10.92	10.85	**10.76**
Qwen2.5-14B	5.29	6.93	5.93	5.87	**5.77**	9.15	10.55	9.58	9.54	**9.45**
Qwen2.5-32B	5.02	6.00	5.41	5.56	**5.31**	8.95	9.78	9.20	9.37	**9.14**
Qwen3-4B	13.66	18.20	**13.96**	14.10	13.98	16.64	20.28	17.27	17.27	**17.07**
Qwen3-8B	9.73	11.86	10.05	10.10	**10.00**	13.30	15.84	13.77	13.77	**13.75**
Qwen3-14B	8.65	10.14	**8.95**	8.96	9.00	12.02	13.63	12.41	12.39	**12.33**
Qwen3-32B	7.61	8.62	8.00	8.09	**7.80**	10.78	12.03	11.15	11.27	**11.04**
Mistral-7B-v0.1	5.25	6.57	5.48	**5.43**	**5.43**	7.74	9.30	8.02	7.98	**7.96**
Mistral-7B-v0.3	5.32	6.38	5.56	5.50	**5.49**	7.83	9.18	8.10	8.05	**8.03**

Table 6: Zero-shot accuracy (avg. over 7 tasks, ↑) and 5-shot MMLU (↑), all 13 models.

	Zero-shot Avg.					5-shot MMLU				
Model	BF16	MXFP4	NVFP4	MicroMix	Ours	BF16	MXFP4	NVFP4	MicroMix	Ours
Llama-2-7B	70.70	66.32	69.32	**69.95**	69.86	45.82	33.57	42.82	**43.66**	41.84
Llama-2-13B	73.75	69.91	72.60	**72.94**	72.86	55.13	45.83	**54.04**	53.63	53.08
Llama-3.1-8B	75.42	67.78	73.29	**73.81**	73.18	65.35	54.73	62.28	**62.83**	62.24
Llama-3.2-3B	68.86	61.29	66.21	**66.79**	66.21	56.72	46.30	52.80	**53.52**	53.20
Qwen2.5-7B	73.63	68.78	72.53	72.57	**73.67**	74.16	66.54	72.18	72.57	**72.72**
Qwen2.5-14B	76.65	74.29	75.39	**76.29**	76.11	79.78	73.12	78.16	78.34	**78.89**
Qwen2.5-32B	77.00	75.70	76.85	76.60	**76.95**	83.21	80.32	82.55	81.63	**82.72**
Qwen3-4B	69.38	63.89	65.14	67.42	**67.48**	70.08	63.32	66.97	67.73	**67.86**
Qwen3-8B	72.47	67.31	71.21	**71.62**	71.43	74.89	68.25	72.75	73.11	**73.60**
Qwen3-14B	76.06	72.51	74.08	75.10	**75.33**	78.76	74.31	77.23	77.71	**77.72**
Qwen3-32B	76.37	73.61	**75.74**	75.65	75.74	81.85	78.69	**81.19**	80.52	80.92
Mistral-7B-v0.1	75.66	71.42	74.63	74.76	**74.79**	62.55	50.71	60.01	60.65	**60.89**
Mistral-7B-v0.3	75.42	70.76	74.18	74.20	**74.80**	62.46	51.74	60.22	60.49	**60.53**

Table 7: Full memory footprint and quantization cost. W-bits is the realized effective weight bit width; memory is stored weight size after quantization; reduction is relative to BF16.

	BF16	MicroMix				AdaptQuant (Ours)			
Model	Mem (GB)	Quant (s)	W-bits	Mem (GB)	Reduc.	Quant (s)	W-bits	Mem (GB)	Reduc.
Llama-2-7B	12.55	144.8	5.37	4.54	2.76×	51.4	4.86	4.21	2.98×
Llama-2-13B	24.24	209.6	5.35	8.52	2.85×	68.9	4.83	7.86	3.08×
Llama-3.1-8B	14.96	157.6	5.37	6.32	2.37×	54.9	5.08	6.14	2.44×
Llama-3.2-3B	5.98	100.5	5.41	2.51	2.38×	41.5	5.12	2.44	2.45×
Qwen2.5-7B	14.19	150.5	5.33	6.08	2.33×	48.7	5.30	6.11	2.32×
Qwen2.5-14B	27.51	249.6	5.31	11.07	2.49×	85.6	5.15	10.95	2.51×
Qwen2.5-32B	61.03	455.4	5.28	22.10	2.76×	131.4	5.08	21.64	2.82×
Qwen3-4B	7.49	135.0	5.40	3.01	2.49×	61.8	5.14	2.93	2.56×
Qwen3-8B	15.26	170.9	5.30	6.61	2.31×	66.9	5.24	6.62	2.31×
Qwen3-14B	27.51	239.5	5.26	10.98	2.51×	81.9	5.19	11.00	2.50×
Qwen3-32B	61.02	421.0	5.25	21.97	2.78×	142.1	5.09	21.68	2.81×
Mistral-7B-v0.1	13.49	159.0	5.36	4.84	2.79×	53.0	5.03	4.64	2.91×
Mistral-7B-v0.3	13.50	157.0	5.36	4.86	2.78×	52.4	5.03	4.65	2.90×

D.3 Inference Throughput

Table 8 gives the full per-configuration data underlying the geomean speedups.

Table 8: Full prefill throughput (tok/s) at sequence length 2048, all 13 models, all tested batch sizes. Parenthesised values are speedup relative to BF16 at the same batch size. OOM marks configurations that exhausted device memory; — marks configurations that did not otherwise complete, and speedup is undefined wherever the BF16 baseline is unavailable.

Model	Method	Batch size				
		1	4	8	16	32
Llama-2-7B	BF16	12 828	13 588	14 028	14 340	14 455
	MicroMix	25 538 (1.99×)	25 443 (1.87×)	25 332 (1.81×)	25 470 (1.78×)	25 513 (1.77×)
	AdaptQuant	25 338 (1.98×)	25 553 (1.88×)	25 560 (1.82×)	25 715 (1.79×)	25 867 (1.79×)
Llama-2-13B	BF16	7 592	7 561	7 598	7 705	OOM
	MicroMix	15 745 (2.07×)	14 706 (1.94×)	14 805 (1.95×)	14 860 (1.93×)	14 878
	AdaptQuant	14 641 (1.93×)	14 415 (1.91×)	14 535 (1.91×)	14 659 (1.90×)	14 760
Llama-3.1-8B	BF16	12 559	12 965	13 327	13 625	13 767
	MicroMix	23 992 (1.91×)	23 979 (1.85×)	24 130 (1.81×)	24 243 (1.78×)	24 266 (1.76×)
	AdaptQuant	23 926 (1.91×)	24 720 (1.91×)	24 825 (1.86×)	24 995 (1.83×)	25 230 (1.83×)
Llama-3.2-3B	BF16	26 242	29 336	29 147	29 833	30 241
	MicroMix	40 310 (1.54×)	46 334 (1.58×)	45 414 (1.56×)	45 658 (1.53×)	45 673 (1.51×)
	AdaptQuant	47 821 (1.82×)	49 409 (1.68×)	48 253 (1.66×)	48 367 (1.62×)	48 440 (1.60×)
Qwen2.5-7B	BF16	14 760	14 624	14 921	14 920	15 065
	MicroMix	26 629 (1.80×)	27 677 (1.89×)	27 272 (1.83×)	27 665 (1.85×)	27 678 (1.84×)
	AdaptQuant	26 812 (1.82×)	27 834 (1.90×)	27 691 (1.86×)	27 850 (1.87×)	28 086 (1.86×)
Qwen2.5-14B	BF16	7 141	7 159	7 203	OOM	OOM
	MicroMix	14 659 (2.05×)	13 944 (1.95×)	14 073 (1.95×)	14 100	14 107
	AdaptQuant	13 402 (1.88×)	13 243 (1.85×)	13 385 (1.86×)	13 489	13 599
Qwen2.5-32B	BF16	OOM	OOM	OOM	OOM	OOM
	MicroMix	5 749	5 636	5 650	5 697	—
	AdaptQuant	6 489	6 604	6 594	—	—
Qwen3-4B	BF16	22 043	21 303	20 910	20 849	21 056
	MicroMix	29 292 (1.33×)	31 708 (1.49×)	30 808 (1.47×)	30 582 (1.47×)	30 519 (1.45×)
	AdaptQuant	34 514 (1.57×)	33 581 (1.58×)	32 432 (1.55×)	32 321 (1.55×)	32 287 (1.53×)
Qwen3-8B	BF16	12 289	12 257	12 487	12 716	12 814
	MicroMix	22 778 (1.85×)	22 741 (1.86×)	22 598 (1.81×)	22 462 (1.77×)	22 437 (1.75×)
	AdaptQuant	22 042 (1.79×)	21 975 (1.79×)	21 944 (1.76×)	22 002 (1.73×)	22 141 (1.73×)
Qwen3-14B	BF16	7 356	7 154	7 120	OOM	OOM
	MicroMix	15 194 (2.07×)	13 900 (1.94×)	13 924 (1.96×)	13 992	13 993
	AdaptQuant	13 596 (1.85×)	13 349 (1.87×)	13 288 (1.87×)	13 629	13 607
Qwen3-32B	BF16	OOM	OOM	OOM	OOM	OOM
	MicroMix	5 427	5 162	5 180	5 205	—
	AdaptQuant	5 921	5 925	5 954	5 842	—
Mistral-7B-v0.1	BF16	12 477	12 972	13 330	13 625	13 749
	MicroMix	24 352 (1.95×)	24 472 (1.89×)	24 602 (1.85×)	24 696 (1.81×)	24 714 (1.80×)
	AdaptQuant	24 003 (1.92×)	24 755 (1.91×)	24 837 (1.86×)	25 013 (1.84×)	25 239 (1.84×)
Mistral-7B-v0.3	BF16	12 597	12 998	13 341	13 609	13 736
	MicroMix	24 360 (1.93×)	24 469 (1.88×)	24 606 (1.84×)	24 725 (1.82×)	24 730 (1.80×)
	AdaptQuant	24 026 (1.91×)	24 756 (1.90×)	24 830 (1.86×)	24 989 (1.84×)	25 217 (1.84×)